```
bescecse@bescecse-HP-Elite-Tower-600-G9-Desktop-PC:~$ spark-shell
25/05/20 11:28:13 WARN Utils: Your hostname, bescecse-HP-Elite-Tower-600-G9-Desktop-PC resolves to a loopback address: 127.0.1.1; using 10.124.3.80 instead (on interface eno1)
25/05/20 11:28:13 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.apache.spark.unsafe.Platform (file:/opt/spark/jars/spark-unsafe_2.12-3.0.3.jar) to constructor java.nio.DirectByteBuffer(long,int)
WARNING: Please consider reporting this to the maintainers of org.apache.spark.unsafe.Platform
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
25/05/20 11:28:13 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://10.124.3.80:4040
Spark context available as 'sc' (master = local[*], app id = local-1747726695958).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.0.3
      /_/

Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 11.0.26)
Type in expressions to have them evaluated.
Type :help for more information.
```

```scala
object WordCountInArray {
  def main(args: Array[String]): Unit = {

    // Example large array input
    val arr = Array(
      "hello world",
      "scala is powerful",
      "hello scala",
      "big data processing",
      "world of scala"
    )

    // Convert array -> words -> count
    val wordCount = arr
      .flatMap(_.split("\\s+"))   // split sentences into words
      .map(_.toLowerCase)        // normalize
      .groupBy(word => word)     // group same words
      .mapValues(_.length)       // count frequency

    // Print result
    wordCount.foreach { case (word, count) =>
      println(s"$word -> $count")
    }
  }
}
```

```
hello -> 2
scala -> 3
world -> 2
```